CL & NLP ASSIGNMENT

Name- Vishnu Choudhary

Sap Id- 500122752

Batch- 04(Data Science)

Question 1 — Numerical: Single-cell forward pass

A scalar LSTM cell receives input $x_t = 0.5$, with previous hidden state $h_{t-1} = 0.2$ and previous cell state $C_{t-1} = 0.4$. The parameters are:

Gate	W	U	b
Forget gate	0.7	0.3	0.1
Input gate	0.6	0.4	0.0
Candidate cell	0.9	0.2	0.0
Output gate	0.5	0.5	0.1

Compute f_t, i_t, $\tilde{C}_t$, C_t, o_t, and h_t.

Ans--

Step 1: Forget gate

The forget gate decides how much of the previous cell memory should be retained.

$f_t = \sigma(Wf\, x_t + Uf\, h_{t-1} + bf)$

$\sigma(z) = 1 / (1 + e^{-z})$

$z_f = (0.7 \times 0.5) + (0.3 \times 0.2) + 0.1 = 0.5100$

$f_t = 1 / (1 + e^{-0.5100}) \approx 0.6248$

So, approximately 62.48% of the previous cell state is retained.

Step 2: Input gate

The input gate controls how much of the proposed new information is written into memory.

$i_t = \sigma(Wi\, x_t + Ui\, h_{t-1} + bi)$

$z_i = (0.6 \times 0.5) + (0.4 \times 0.2) + 0.0 = 0.3800$

$i_t = 1 / (1 + e^{-0.3800}) \approx 0.5939$

The input gate allows approximately 59.39% of the candidate information through.

Step 3: Candidate cell state

The candidate state represents new information that could be added to the cell memory.

$\tilde{C}_t = \tanh(Wc\, x_t + Uc\, h_{t-1} + bc)$

$z_c = (0.9 \times 0.5) + (0.2 \times 0.2) + 0.0 = 0.4900$

$\tilde{C}_t = \tanh(0.4900) \approx 0.4542$

Step 4: Updated cell state

The updated cell state combines the retained old memory with the selected candidate information.

$C_t = f_t\, C_{t-1} + i_t\, \tilde{C}_t$

$C_t = (0.6248 \times 0.4) + (0.5939 \times 0.4542)$

$C_t \approx 0.2499 + 0.2697 = 0.5196$

Step 5: Output gate

The output gate controls how much of the cell memory is exposed as the hidden state.

$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$

$z_o = (0.5 \times 0.5) + (0.5 \times 0.2) + 0.1 = 0.4500$

$o_t = 1 / (1 + e^{-0.4500}) \approx 0.6106$

Step 6: Hidden state

$h_t = o_t \tanh(C_t)$

$h_t = 0.6106 \times \tanh(0.5196) \approx 0.6106 \times 0.4774$

The calculated hidden state is approximately 0.2915.

Quantity	Value
Forget gate f_t	0.6248
Input gate i_t	0.5939
Candidate $\tilde{C}_t$	0.4542
Cell state C_t	0.5196
Output gate o_t	0.6106
Hidden state h_t	0.2915

Question 2 — Numerical: LSTM parameter count

An LSTM layer takes 10-dimensional input vectors and has 32 hidden units. (a) Find the number of trainable parameters using one bias vector per gate. (b) Find the number using two bias vectors per gate, as in PyTorch nn.LSTM.

An LSTM has four gate transformations: forget, input, candidate, and output. Each gate has input weights, recurrent weights, and bias terms.

Ans--

Part (a): One bias vector per gate

Input weights per gate = $H \times D = 32 \times 10 = 320$

Recurrent weights per gate = $H^2 = 32 \times 32 = 1{,}024$

Biases per gate = $H = 32$

Parameters per gate = $320 + 1{,}024 + 32 = 1{,}376$

Total = $4 \times 1{,}376 = 5{,}504$

The layer has 5,504 trainable parameters under the one-bias convention.

Part (b): Two bias vectors per gate

Under this convention, each gate has two bias vectors, each containing 32 values.

Biases per gate = $2H = 2 \times 32 = 64$

Parameters per gate = $320 + 1{,}024 + 64 = 1{,}408$

Total = $4 \times 1{,}408 = 5{,}632$

The layer has 5,632 trainable parameters under the two-bias convention.

Component	One bias per gate	Two biases per gate
Input weights	1,280	1,280
Recurrent weights	4,096	4,096
Biases	128	256
Total	5,504	5,632

The difference is 128 parameters because the second convention adds one extra 32-value bias vector for each of the four gates.

Question 3 — Numerical: Vanishing gradient through the cell state

The gradient flowing backwards through the cell state is approximated by f^k when the forget gate has a constant value f. (a) Find the scaling factor for f = 0.9 after 10 and 50 steps. (b) For 50 steps, find it for f = 0.5 and f = 0.99. (c) Explain what the results mean for learning long-term dependencies.

Gradient scaling factor: $S = f^k$

Ans--

Part (a): f = 0.9

After 10 steps: $S = 0.9^{10} = 0.348678\ldots \approx 0.3487$

About 34.87% of the gradient remains after 10 steps.

After 50 steps: $S = 0.9^{50} \approx 0.0051538 \approx 0.0052$

Only about 0.52% of the gradient remains after 50 steps.

Part (b): 50 steps

For f = 0.5: $S = 0.5^{50} \approx 8.8818 \times 10^{-16}$ (effectively zero)

For f = 0.99: $S = 0.99^{50} \approx 0.6050$

When the forget gate is close to 1, much more of the gradient survives. When it is smaller, repeated multiplication causes the gradient to shrink rapidly.

Forget gate f	Steps k	Scaling factor
0.9	10	0.3487
0.9	50	0.0052
0.5	50	8.8818×10^{-16}
0.99	50	0.6050

Part (c): Explanation

When learning a dependency 50 time steps back, a forget gate of 0.9 leaves only about 0.52% of the gradient, making the relationship difficult to learn. Compared with a vanilla RNN, an LSTM uses a cell state with additive updates and learnable gates, which can help information and gradients pass through many time steps more effectively. This advantage is not guaranteed if the forget gates repeatedly take small values.

Question 4 — Real-world scenario: Electricity load forecasting

A utility has hourly electricity-load records for three full 365-day years, with no missing data. An LSTM uses a 168-hour lookback and three features per hour (load, temperature, holiday flag) to predict the next 24 hours. The stride is one hour.

The questions ask for the number of records and windows, input/output batch shapes, training windows and mini-batches for the first 70% of records, and parameter count for a hidden size of 64.

Part (a): Number of hourly records

Number of days = $3 \times 365 = 1{,}095$

Hourly records = $1{,}095 \times 24 = 26{,}280$

There are 26,280 hourly records.

Part (b): Number of windows

Each sample needs 168 input hours and 24 future target hours. With a stride of one, the number of complete windows is:

$W = N - L - Hf + 1$

$W = 26{,}280 - 168 - 24 + 1 = 26{,}089$

There are 26,089 input/output windows. The +1 includes the final valid window that ends exactly at the last record.

Part (c): Batch tensor shapes

The usual LSTM input layout is (batch size, sequence length, number of features).

Input batch X: (64, 168, 3)

Each window predicts 24 future hourly load values, so a scalar target per future hour gives:

Output batch Y: (64, 24)

If the implementation keeps a separate singleton feature axis, the output shape can instead be (64, 24, 1).

Part (d): Training windows and mini-batches

Training records = $0.70 \times 26{,}280 = 18{,}396$

Generate training windows only inside this training segment so no training target crosses into the validation segment.

Training windows = $18{,}396 - 168 - 24 + 1 = 18{,}205$

Mini-batches = ceil(18,205 / 64) = 285

There are 284 full batches (64 windows each) and one final partial batch of 29 windows.

There are 18,205 training windows and 285 mini-batches per epoch.

Part (e): LSTM parameter count

For input dimension D = 3, hidden size H = 64, four gates, and one bias vector per gate:

$P = 4(HD + H^2 + H)$

$P = 4[(64 \times 3) + (64 \times 64) + 64]$

$P = 4(192 + 4{,}096 + 64) = 17{,}408$

The LSTM layer has 17,408 trainable parameters. A separate forecasting head is not included.

Part	Answer
(a) Hourly records	26,280
(b) Windows	26,089
(c) Input batch	(64, 168, 3)
(c) Output batch	(64, 24) or (64, 24, 1)

(d) Training windows	18,205
(d) Mini-batches	285
(e) LSTM parameters	17,408

Question 5 — Real-world scenario: Predictive maintenance

A factory monitors a pump using a vibration sensor. Calibration readings ranged from 20 to 120 mm/s. The LSTM uses the last 50 cycles to predict whether the pump will fail in the next cycle.

The question asks you to normalise a reading of 95 mm/s, classify a failure probability of 0.83 using a threshold of 0.5, calculate a confusion matrix and evaluation metrics, and recommend a metric for maintenance.

Part (a): Min-max normalisation

$x' = (x - x_{min}) / (x_{max} - x_{min})$

$x' = (95 - 20) / (120 - 20) = 75 / 100 = 0.7500$

The normalised reading is 0.75.

Part (b): Failure prediction

The predicted failure probability is 0.83, which is above the decision threshold of 0.5.

$0.83 \geq 0.5 \rightarrow$ predict failure

The model predicts that the pump will fail within the next cycle.

Part (c): Confusion matrix

There are 1,000 cycles, 40 actual failures, 45 predicted failures, and 30 correct failure flags.

True positives (TP) = 30

False positives (FP) = 45 − 30 = 15

False negatives (FN) = 40 − 30 = 10

Actual non-failures = 1,000 − 40 = 960

True negatives (TN) = 960 − 15 = 945

	Actual failure	**Actual no failure**	**Total**
Predicted failure	TP = 30	FP = 15	45
Predicted no failure	FN = 10	TN = 945	955
Total	40	960	1,000

Precision

Precision = TP / (TP + FP) = 30 / 45 = 0.6667 = 66.67%

Of the cycles flagged as failures, 66.67% were actual failures.

Recall

Recall = TP / (TP + FN) = 30 / 40 = 0.7500 = 75.00%

The model catches 75% of the actual failures and misses 25%.

F1-score

F1 = 2TP / (2TP + FP + FN)

F1 = 60 / (60 + 15 + 10) = 60 / 85 = 0.7059 = 70.59%

Accuracy

Accuracy = (TP + TN) / Total

Accuracy = (30 + 945) / 1,000 = 0.9750 = 97.50%

Metric	Calculation	Result
True positives	Correct failure flags	30
False positives	45 − 30	15
False negatives	40 − 30	10
True negatives	960 − 15	945
Precision	30 / 45	66.67%
Recall	30 / 40	75.00%
F1-score	60 / 85	70.59%
Accuracy	975 / 1,000	97.50%

Part (d): Which metric should be prioritised?

Recall is usually the first priority in predictive maintenance because a missed failure can lead to expensive equipment damage, downtime, or safety risks. Precision also matters because too many false alarms can cause unnecessary inspections and costs.

Accuracy can be misleading when classes are imbalanced. Only 40 of 1,000 cycles (4%) are followed by failure, while 960 (96%) are non-failures. A model that always predicts “no failure” would still achieve 96% accuracy while detecting none of the actual failures.

The team should prioritise recall when missed failures are costly, while monitoring precision and choosing a threshold that balances operational risk and inspection costs.